Architecture Design Document
DCC-EX Line Side Sound

Design and Development

THIS IS NOT A TOY

Table of Contents

1 Overview 6

2 Processing Model: Asymmetric Multiprocessing (AMP) 8

2.1 The High-Speed real-time Audio Pipeline (Data Stages) 8

2.2 Flow control and back pressure 10

2.2.1 The Pacemaker: I²S Hardware -> Matrix Mixer (Core 1) 10

2.2.2 Matrix Mixer -> MP3 Decoder (Core 1) 11

2.2.3 MP3 Decoder -> Tier 4 SRAM (Core 1 -> Core 0) 11

2.2.4 Core 0 -> Tier 4 SRAM 11

2.2.5 PSRAM -> SD Card (Core 0) 12

2.3 Data burst and PCM ring-buffer overruns 12

2.3.1 The Static Holding Tank (pcm_buf) 12

2.3.2 Sipping from the Tank 12

2.3.3 The Refill Trigger 13

2.4 Matrix mixer 13

2.5 Core to Core Communication & Synchronization 13

2.6 Core 0: System Logistics & Supervision 15

2.7 Core 1: Deterministic DSP Engine 15

3 Memory Architecture 16

3.1 SRAM Memory Mapping & Bus Isolation 16

4 Control & Communication 16

4.1 PIO-Based I²C Bridge 16

4.2 Multi-Byte Writes (File IDs) 16

5 Target I²C Registers 17

5.1 Global I²C Registers 17

5.1.1 Register 0x00 Write only SLOT_SELECTOR 18

5.1.2 Register 0x01 Write only GLOBAL_CMD bitmask 18

5.1.3 Register 0x02 Read only SYS_STATUS bitmask 19

5.1.4 Register 0x03 Read only FW_VERSION_MAJOR 19

5.1.5 Register 0x04 Read only FW_VERSION_MINOR 19

5.1.6 Register 0x05 R/W OLED_PAGE 19

5.1.7 Register 0x06 Read only Core 0 peak load 20

5.1.8 Register 0x07 Read only Core 1 peak load 20

5.1.9 Register 0x08 R/W SCRIPT_ENGINE 20

5.1.10 Register 0x09 R/W SCRIPT_LSB 20

5.1.11 Register 0x0A R/W SCRIPT_MSB 21

5.1.12 Register 0x0B R/W SCRIPT_CMD_TRIGGER 21
5.1.13 Register 0x0C R/O TEMP_LSB 21
5.1.14 Register 0x0D R/O TEMP_MSB 21
5.2 Indexed I²C Registers (Slot-Specific) 22
5.2.1 I²C Playback & Volume Control registers 22
5.2.2 Register 0x10 [indexed] R/W PLAY_CONTROL 22
5.2.3 Register 0x11 [indexed] Read only PLAY_STATUS 22
5.2.4 Register 0x12 through 0x19 VOL_OUT (Read/Write) 22
5.3 I²C Sound File Management & Status registers 23
5.3.1 Register 0x20 [indexed] R/W FILE_ID_LSB 23
5.3.2 Register 0x21 [indexed] R/W FILE_ID_MSB 24
5.3.3 Register 0x22 [indexed] Write Only SLOT_COMMAND 24
5.3.4 Register 0x23 [indexed] Read only SLOT_STATUS 24
5.3.5 Register 0x24 [indexed] Read only SLOT_ERROR bitmask slot error 24
5.3.6 Register 0x25 [indexed] Read only FILE_SAMPLE_RATE 25
5.3.7 Register 0x26 [indexed] Read only FILE_BITRATE 25
5.3.8 Register 0x27 [indexed] Read only FILE_CHANNELS 25
5.4 I²C Fading registers 25
5.4.1 Register 0x28 [indexed] RW FADE_CHANNEL 26
5.4.2 Register 0x29 [indexed] RW FADE_TARGET_VOL 26
5.4.3 Register 0x2A [indexed] RW FADE_ DURATION_LSB 26
5.4.4 Register 0x2B [indexed] RW FADE_ DURATION_MSB 26
5.4.5 Register 0x2C [indexed] Write only FADE_TRIGGER 26
5.5 I²C Scripting engine telemetry registers 27
5.5.1 Registers 0x30 – 0x4F Read only SCRIPT_ENGINES Telemetry 27
6 User interface CLI commands 28
6.1 Introduction 28
6.2 Global Write Commands (<D SYS ...>) 28
6.3 Slot Write Commands (<D SLOT [slot] ...>) 30
6.4 Status & Read Commands (<R ...>) 31
7 Scripting Commands 35
7.1 Invalid Script Commands 35
7.2 Valid CLI Commands for Scripts 36
8 Crash Data Logging and Recovery Architecture 37
8.1 Storage Layout in Flash EEPROM 37
8.2 CPU Hard Fault Interception (VTOR Redirection) 37

8.3 Hardware Watchdog Resets (0xDEADBEEF) 38
8.4 The "Three Strikes" Safe Mode Safety Net 38
8.5 Automatic Strike Counter Reset 39

Table of Figures

Figure 1 Overall system architecture design 7
Figure 2 High-Speed real-time Audio Pipeline 9
Figure 3 I²C Registers 17

Table of Tables

Table 1 Revision history 5
Table 2 Memory Architecture 16
Table 3 Custom Linker Script SRAM Allocation 16
Table 4 I²C register overview 18
Table 5 Register 0x00 SLOT_SELECTOR 18
Table 6 Register 0x01 GLOBAL_CMD 19
Table 7 Register 0x02 SYS_STATUS 19
Table 8 Register 0x03 FW_VERSION_MAJOR 19
Table 9 Register 0x04 FW_VERSION_MINOR 19
Table 10 Register 0x5 OLED_PAGE 20
Table 11 Register 0x06 Core 0 peak load in % 20
Table 12 Register 0x07 Core 1 peak load in % 20
Table 13 Register 0x08 Auto scripting engine selection 20
Table 14 Register 0x09 Automation ScriptID LSB 20
Table 15 Register 0x0A Automation ScriptID MSB 21
Table 16 Register 0x0B Command trigger to Run or Stop script 21
Table 17 Register 0x0C TEMP_LSB 21
Table 18 Register 0x0C TEMP_LSB 21
Table 19 Register 0x10 [indexed] PLAY_CONTROL 22
Table 20 Register 0x11 [indexed] PLAY_STATUS 22
Table 21 Register 0x12 [indexed] VOL_OUT channel 0 23
Table 22 Register 0x13 [indexed] VOL_OUT channel 1 23
Table 23 Register 0x14 [indexed] VOL_OUT channel 2 23
Table 24 Register 0x15 [indexed] VOL_OUT channel 3 23
Table 25 Register 0x16 [indexed] VOL_OUT channel 4 23
Table 26 Register 0x17 [indexed] VOL_OUT channel 5 23
Table 27 Register 0x18 [indexed] VOL_OUT channel 6 23
Table 28 Register 0x19 [indexed] VOL_OUT channel 7 23
Table 29 Register 0x20 [indexed] FILE_ID_LSB 23
Table 30 Register 0x21 [indexed] FILE_ID_MSB 24
Table 31 Register 0x22 [indexed] SLOT_COMMAND 24

Table 32 Register 0x23 [indexed] SLOT_STATUS 24
Table 33 Register 0x24 [indexed] SLOT_ERROR 24
Table 34 Register 0x25 [indexed] FILE_SAMPLE_RATE 25
Table 35 Register 0x26 [indexed] FILE_BITRATE 25
Table 36 Register 0x27 [indexed] FILE_CHANNELS 25
Table 37 Register 0x28 [indexed] FADE_CHANNEL 26
Table 38 Register 0x29 [indexed] FADE_TARGET_VOL 26
Table 39 Register 0x2A [indexed] FADE_DURATION_LSB 26
Table 40 Register 0x2B [indexed] FADE_DURATION_MSB 26
Table 41 Register 0x2C [indexed] FADE_TRIGGER 26
Table 42 Register 0x30-0x4F Scripting Engines telemetry 27

Version	Changes	Date
0.1	Initial draft	2026-07-25
0.2	Reviewed and corrections between code base and document	2026-07-26
0.3	Minor adjustments	2026-07-26
0.3	Added crash data logging and recovery architecture	2026-08-12
0.4	Added support for RP2350B internal temperature sensor	2026-08-22

Table 1 Revision history

1 Overview

The primary objective of this project is to develop a robust, line side sound system for model railway layouts that replaces the legacy architecture of I²C-connected UARTs and DFPlayer MP3 modules. The current solution relied on expensive NXP SC16IS752 dual UARTs and individual DFPlayer boards for each speaker, which lacked remote reset capabilities and required extensive cabling.

By moving to a centralized Raspberry Pi RP2350 microcontroller, the system gains I²S outputs, remote software reset capability via EX Rail command script, and a sophisticated matrix mixer to route audio streams dynamically across multiple output channels.

Calculations show that when the source MP3 files are sampled at 22,050 Hz (MPEG-2 Layer III), mono, bitrate 128Kbps, the highest frequency is about 11 Khz, which is more than enough for line side quality samples. The microSD card transfer rate for 128Kbps bit rate translates to about 16Kbyte/sec. With 8 simultaneous streams, only 128Kbyte/sec is required, very safe for the core that handles reading the microSD card. Achievable microSD card reads via SPI is between 1,500 to 2,000 Kbyte/s. However reading multiple files via the FatFS system is very slow, therefor we build a sector table for each file during load-time, enabling the background loop to use high-speed block reads, bypassing FatFs filesystem seek overhead during playback.

Features now possible with this solution:

1. Direct I²C end-to-end control with EX Rail scripting
2. Only a single microSD card per board, there can be multiple boards on an I²C bus
3. Software reset of system from the Command Station via EX Rail command
4. Extensive telemetry from the system, such as error codes for MP3 file invalid formats (wrong sampling, unsupported bitrate, not mono etc.)
5. Handling microSD card removal or insertion without going offline at the I²C level
6. Matrix mixer to mix any MP3 streams to any output channel simultaneously
7. Route one or more MP3 streams to more than one output, sound samples can dynamically move between outputs.
8. Fading engine, rich flexible fading-in and fading-out features
9. Scripting engine, handling 4 simultaneous scripts, each script can be controlled from an EXRail script, real time script execution status.
10. EX Rail script controlled mixer audio volume, fading in and out, creating a sense of 3D sound on a layout.
11. User updatable firmware via USB.

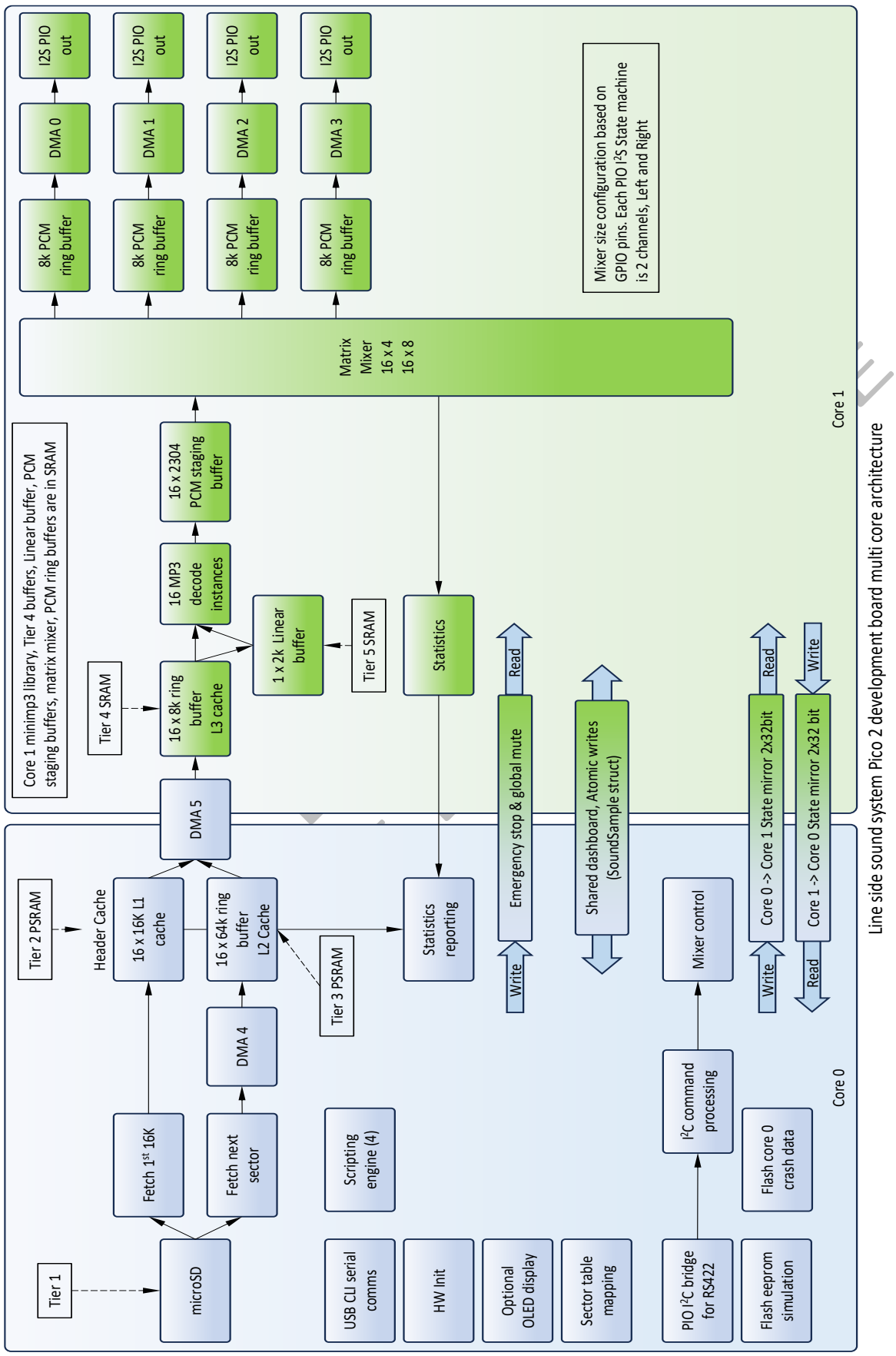

Figure 1 Overall system architecture design

Line side sound system Pico 2 development board multi core architecture

2 Processing Model: Asymmetric Multiprocessing (AMP)

The DCC-EX LSS Engine is an asymmetric multi-core audio processor designed to play up to 16 concurrent MP3 streams mixed into 8 physical output channels. The design prioritizes deterministic timing for the audio engine (Core 1) by isolating it from the non-deterministic "logistics" tasks (Core 0), such as SD card access and I^2C communication.

Core 0: Logistics & Supervision (Manager)

- Filesystem management (FatFs) and SD Card SPI communication.
- I^2C Target interface via PIO (I^2C-to-RS422 bridge).
- PSRAM "Holding Tank" management and allocation.
- Pushing data from PSRAM to Tier 4 Exchange buffers via M2M DMA.
- USB CLI and system telemetry reporting.

Core 1: Deterministic DSP Engine (Worker)

- Multi-instance MP3 decoding (16x minimp3).
- 16x8 Matrix mixing with individual volume scalars.
- PCM saturation and 2x Zero-Order Hold up-sampling.
- Feeding the PCM output ring buffers for autonomous I^2S FIFO DMA.

The used open source libraries are:

- Minimp3, library is trimmed to save on SRAM space so only functions and decoding tables needed to decode MP3 sound sample that have a bit-rate of 128Kbps, are sampled at 22050Hz and Mono are included.
- no-OS-FatFS-SD-SDIO-SPI-RPi-Pico, SPI interface is used

2.1 The High-Speed real-time Audio Pipeline (Data Stages)

The pipeline is divided into four distinct stages to bridge the gap between the high-latency SD card and the zero-latency I^2S output requirements.

Stage 1: SD Card to PSRAM (The Holding Tank)

- Header Cache (16KB per stream): The first 16KB of every sound file is stored in PSRAM. This enables "Instant-Start" playback, bypassing SD seek latency. Since the header cache is not refreshed when a file has finished playing we can instantly play again, or when playing in a loop it is seamless.
- Sector Map: Core 0 generates a map of physical sectors during load-time. This allows the background loop to use high-speed block reads, bypassing FatFs filesystem overhead during playback.
- PSRAM Ring Buffer (64KB per stream): A large buffer used to top up data ahead of the CPU, absorbing SD card read-stutters.

Stage 2: PSRAM to Tier 4 Exchange (The Air Gap)

- Location: SRAM Banks 3 and 4.
- Size: 128KB (16x 8KB buffers).

- Mechanism: Core 0 proactively monitors the atomic updated by Core 1 ‘stream_read_pointers’. When Core 0 detects that an 8KB Tier 4 buffer is half-empty, it autonomously triggers a high-speed Memory-to-Memory DMA to "blast" data from PSRAM into SRAM, without Core 1 ever needing to ask.
- Bank Isolation: Because these reside in dedicated SRAM banks, Core 0's DMA writes do not stall Core 1's decoder reads.

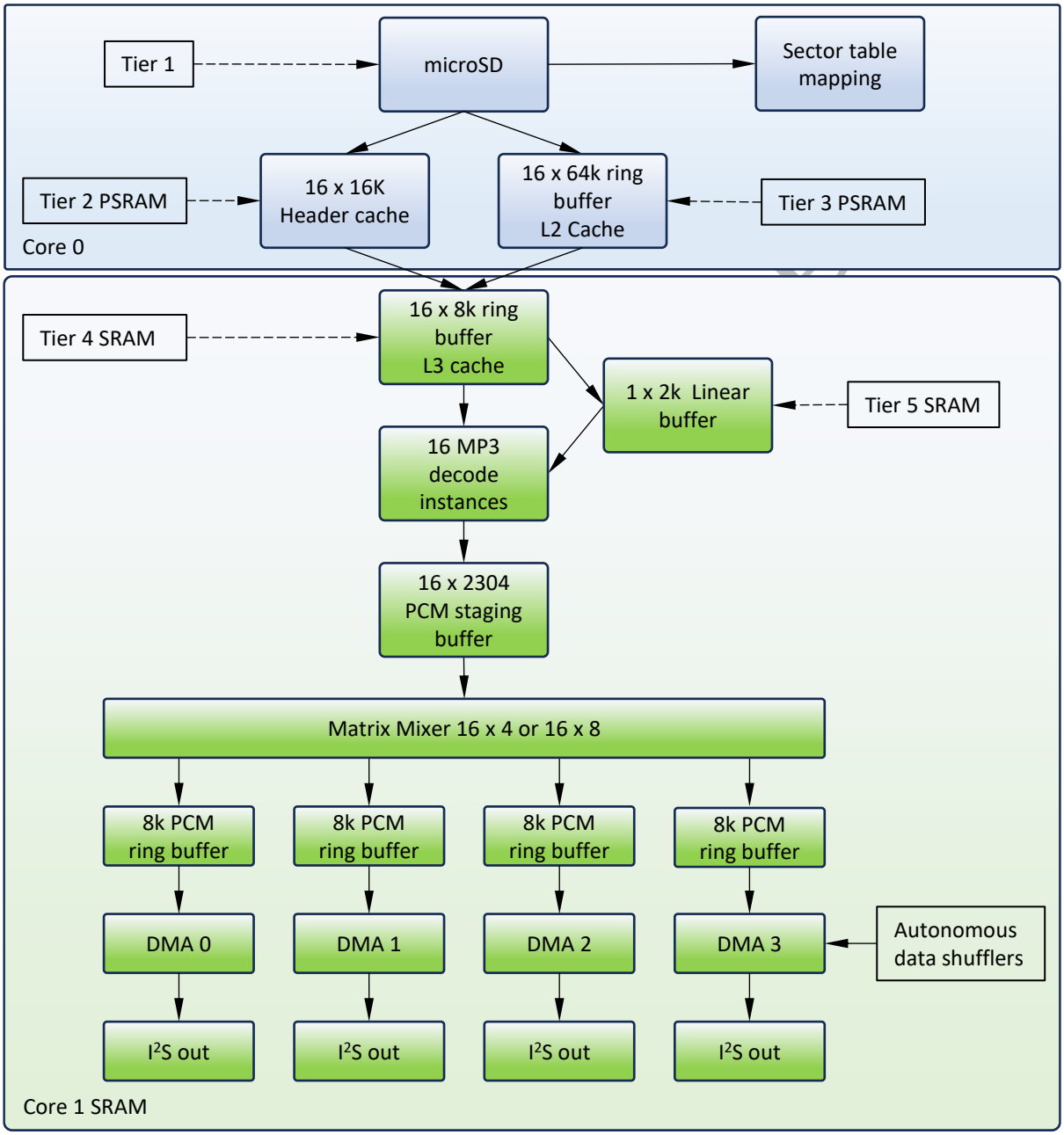

Figure 2 High-Speed real-time Audio Pipeline

Stage 3: Tier 4 to Decoder (Linearization)

- Linear buffer (2KB): MP3 frames often span the wrap-around boundary of a ring buffer.
- Logic: If Core 1 detects a frame is split, it copies the tail and head fragments into this 2KB linear buffer. The decoder is then pointed to the linear buffer, ensuring it always sees a contiguous bitstream.

Stage 4: Matrix Mixer to PCM Output (Autonomous Shuffling)

- Matrix Mixer: Up to 16 streams are mixed to 16-bit PCM, scaled by an 8-bit volume factor, and summed into 32-bit accumulators.
- PCM Output Buffers: Located in SRAM Bank 2, aligned to 8192-byte boundaries.
- Autonomous DMA: The DMA hardware is configured with "Ring Wrapping" (2^13) and to run indefinitely.
  Because the buffers are physically aligned and the ring size is set in hardware, the DMA read pointers automatically wrap from the end of the buffer back to the start with zero CPU intervention or boundary checking.
- PIO I^2S Interface: Data is fed into PIO FIFO state machines, which drive the physical amplifiers with bit-perfect timing. The PIO I^2S state machines run in lockstep.

2.2 Flow control and back pressure

The architecture employs a strict, mathematically bound "Pull-Based" (Demand-Driven) Pipeline. The data is never pushed from the top down; it is exclusively pulled from the bottom up. The ultimate pacemaker of the entire system is the physical hardware clock to the I^2S Amplifier.
Tracing the backpressure mechanism backward from the speaker to the SD Card to see exactly how overruns are prevented at every single boundary.

Because the 50% threshold on the Tier 4 buffer is purely a "Request for Data" signal, and the actual fulfilment of that data is bounded by a strict ‘space_avail’ capacity calculation, buffer overruns are physically impossible by design.
The entire pipeline acts like a perfectly synchronized bucket brigade, where no one hands a bucket backward until the person behind them explicitly says they have room for it.

2.2.1 The Pacemaker: I^2S Hardware -> Matrix Mixer (Core 1)

The physical I^2S BCLK/LRCLK pins dictate that exactly 44,100 stereo sample pairs (88,200 16-bit words) must be consumed per second.

- Core 1 does not constantly run its matrix mixer.
- At the start of the ‘while(true) loop’, Core 1 calculates exactly how far the hardware DMA has moved (free_words = dma_read_ptr - sw_write_ptr).
- **The Backpressure:** If free_words < 64, Core 1 hits tight_loop_contents() and yields. It physically refuses to mix more audio or decode more MP3 frames until the physical speaker has consumed enough data.

2.2.2 Matrix Mixer -> MP3 Decoder (Core 1)

Inside the mixer loop, Core 1 pulls individual 16-bit PCM samples out of the decoder's holding array (stream_states[i].pcm_buf) via the read_pos index.

- **The Backpressure:** The minimp3 decoder is never allowed to run autonomously. Core 1 strictly evaluates if (read_pos >= valid_samples).
- Only when the previous MP3 frame's PCM data has been 100% consumed by the matrix mixer does Core 1 command minimp3 to decode the next frame. The decoder never generates data into a buffer that isn't completely empty.

2.2.3 MP3 Decoder -> Tier 4 SRAM (Core 1 -> Core 0)

When the decoder does run, it consumes a few hundred bytes of compressed MP3 data from the Tier 4 Exchange Ring Buffer, autonomously advancing stream_read_pointers[i]

- **The Refill Trigger (Proactive Polling):** Core 1 never asks for data. Instead, Core 0 continuously evaluates the distance between its own write pointer and Core 1's read pointer in its main logistics loop.
- **The 50% Threshold:** When Core 0 calculates that the remaining unread compressed data has fallen below 50% (bytes_avail < TIER4_BUFFER_SIZE / 2), it autonomously triggers the Memory-to-Memory DMA to blast the next 4KB of data from PSRAM into the Tier 4 SRAM buffer.

2.2.4 Core 0 -> Tier 4 SRAM

When Core 0's proactive polling detects that a stream's buffer has fallen below the 50% threshold, it must top it up without overrunning Core 1's read pointer.

- **The Backpressure (Static Safety):** Core 0 does not calculate a dynamic remaining space. Because it strictly waits until the buffer is *less than* half full (bytes_avail < TIER4_BUFFER_SIZE / 2), it mathematically guarantees that exactly 50% of the buffer (4096 bytes) is physically empty and safe to write to.
- **The Clamp:** It simply sets space_avail = TIER4_BUFFER_SIZE / 2. It then clamps this value to ensure it doesn't exceed the remaining file size or cross a physical memory boundary (like the end of the PSRAM cache).
- **The Transfer:** Core 0 fires a hardware DMA transfer of exactly that clamped amount. If Core 1 were to stop consuming data entirely (e.g., during a pause or error), the buffer would never drop below 50%, the trigger condition would never be met, and Core 0 would safely transfer nothing. It is mathematically impossible for Core 0 to overwrite data in Tier 4.

2.2.5 PSRAM -> SD Card (Core 0)

As Core 0 drains data from the 64KB PSRAM ring buffer to feed Tier 4, the bytes_in_ring value decreases.

- **The Backpressure:** During its background tasks, Core 0 checks ring_free = ring_buffer_size - bytes_in_ring;.
- It will only ever command the SD Card to read the next chunk if ring_free >= 16384 (the exact size of a chunk). It never guesses; it guarantees the space exists before the sd_read_blocks command is even issued.

2.3 Data burst and PCM ring-buffer overruns

An MP3 frame decoding is a highly "bursty" operation, you feed it a few hundred bytes, and it instantly spits out thousands of bytes of raw PCM audio. However, the architecture completely sidesteps this danger because minimp3 does not output into the PCM ring buffer.

2.3.1 The Static Holding Tank (pcm_buf)

When Core 1 commands minimp3 to decode a frame, the output does not go into the DMA ring buffer. Instead, it goes into a dedicated, linear scratchpad array (PCM staging buffer) defined in core_1.cpp inside the Core1StreamState struct:

```
struct Core1StreamState {
    int16_t pcm_buf[MINIMP3_MAX_SAMPLES_PER_FRAME]; // Up to 2304 samples
    int valid_samples;
    int read_pos;
    int channels;
};
```

The macro MINIMP3_MAX_SAMPLES_PER_FRAME is hardcoded inside the minimp3 library to 2304 samples. 2304 is the absolute theoretical maximum number of samples that any MP3 frame can possibly decompress into (specifically, MPEG-1 Layer 3 Stereo, which generates 1152 samples per channel). Since the engine strictly enforces MPEG-2 Layer 3 Mono (which expands to exactly 576 samples), the pcm_buf is heavily over-provisioned. It is physically impossible for the decoder to generate more data than this static array can hold.

2.3.2 Sipping from the Tank

Once the decoder fires and fills pcm_buf with 576 samples, the matrix mixer does not dump them all into the hardware I^2S ring buffer at once. Instead, the matrix mixer "sips" from this holding tank in tiny, mathematically strict increments governed by CHUNK_PAIRS (16 source samples per iteration):

1. **The Pace:** The mixer checks the I^2S DMA Ring Buffer: "Do I have at least 64 free words of space?" (free_words >= CHUNK_WORDS).
2. **The Sip:** If yes, it pulls exactly 16 mono samples out of pcm_buf (using read_pos++).

3. **Stereo Duplication & Mix:** It instantly duplicates each mono sample into a Stereo (L/R) pair and applies the active volume scalars.
4. **The Up-sample:** It writes each of those mixed L/R pairs into the accumulators twice (a 2x zero-order hold up-sample) so the I^2S amplifier consume standard 44.1Khz samples.
5. **The Output:** 16 source samples × 2 (Stereo) × 2 (Up-sample) perfectly yields the 64 words, which are then saturated and committed to the I^2S Ring Buffer.

2.3.3 The Refill Trigger

The minimp3 decoder is only ever allowed to fire again once read_pos >= valid_samples (meaning the matrix mixer has completely consumed all 576 samples from the previous frame).

Because the Matrix Mixer only consumes data when the hardware I^2S DMA explicitly has room for it, and the decoder only fires when the holding tank is completely empty, the bursty nature of MP3 decoding is perfectly smoothed out, and a buffer overrun is mathematically impossible

2.4 Matrix mixer

The Matrix Mixer serves as the computational heart of the audio engine, providing a high-density routing fabric that allows each of the 16 decoded input streams to be mapped to any or all of the 8 physical mono output channels simultaneously.

During every DSP cycle, Core 1 iterates through each active stream and applies eight independent 8-bit volume scalars, one for each hardware speaker, to the incoming PCM data. This "all-to-all" architecture enables complete spatial flexibility; for example, a single sound can have its volume set to maximum for all eight outputs, effectively broadcasting the sound across the entire layout so it is heard on every speaker at once.

To maintain audio fidelity during this intensive summation, the mixer utilizes 32-bit signed accumulators to prevent digital clipping during the mixing phase, followed by a hardware-accelerated 16-bit saturation stage using ARM DSP intrinsics before the final data is handed off to the I^2S buffers.

2.5 Core to Core Communication & Synchronization

- **Lock-Free Pointers:** All data movement relies on atomic stream_read_pointers and stream_write_pointers. No mutexes or spinlocks are permitted on Core 1 to ensure 100% deterministic execution.
- **The State Mirror (No Queues):** Inter-core control and telemetry do not use queues. Instead, they rely on two lock-free arrays (core0_to_core1_cmd[2] and core1_to_core0_status[2]). Each array consists of two 32-bit words, where each of the 16 audio streams is assigned a strict 4-bit nibble. Core 1 instantly changes its specific 4-bit nibble to report its playback state or an error, and Core 0 safely reads it during its logistics loop. Because the RP2350 is a 32-bit architecture, dividing the 64 bits of total state across two 32-bit words guarantees that every status update is a perfectly atomic, single-cycle memory operation.
- **Emergency Stop (Global Command Mirror):** The "Emergency Stop" signal (e.g., if the SD card is removed) does not use a hardware FIFO or IRQ. Instead, Core 0 sets

the GCMD_EMERGENCY_STOP bit in the lock-free global_cmd_mirror. Core 1 snapshots this mirror at the exact start of every DSP cycle. If it sees the stop bit, it instantly zeroes all volumes and read pointers, safely halting audio.

- **I²C Wire-Tap:** A 64KB lock-free circular buffer in PSRAM records all incoming/outgoing I²C traffic, allowing real-time deep diagnostics via the USB CLI without ever stalling the I²C PIO State Machines.

Core 0 uses the command mirror to instruct Core 1 on what it wants each specific audio stream to do.

- **CMD_IDLE (0x0):** Stop playing. Core 1 will gracefully fade out and stop decoding.
- **CMD_PLAY (0x1):** Start or continue playing the stream normally.
- **CMD_FLUSH (0x2):** Instantly abort playback, zero out all tracking pointers, and flush the mixer state without waiting for a fade-out.
- **CMD_LOOP (0x3):** Play the stream continuously. Core 1 will treat it as playing and expect Core 0 to instantly wrap the data buffers upon EOF.
- **CMD_PAUSE (0x4):** Freeze the stream. Core 1 stops pulling samples and mixing, but perfectly maintains its read pointers so it can instantly resume.

Core 1 uses its command mirror to provide real-time telemetry back to Core 0. Core 0 reads this to know if Core 1 successfully executed a command, or if something went wrong.

- **STAT_ACK_IDLE (0x0):** Core 1 acknowledges that it has stopped and sitting idle. (Also sent to acknowledge a FLUSH command).
- **STAT_ACK_PLAYING (0x1):** Core 1 acknowledges a PLAY or LOOP command and confirms it is actively decoding and mixing audio.
- **STAT_EOF_FADING (0x2):** Core 1 hit the end of the file data (EOF) or was told to stop, and is currently performing a quick, natural volume fade-out (slew) to prevent speaker pops. It will transition to STAT_ACK_IDLE when the fade is complete.
- **STAT_ERROR (0x3):** Core 1 encountered a fatal decoding error (e.g., Variable Bit Rate detected, corrupted sync word, or Stereo data). It halts playback and waits for Core 0 to clear the error.
- **STAT_ACK_PAUSED (0x4):** Core 1 acknowledges a PAUSE command and confirms the stream is frozen.

There is also a Global Command Mirror. Instead of an array, this is a single 32-bit integer that affects the entire DSP engine at once.

- **GCMD_EMERGENCY_STOP (Bit 0):** This is the ultimate kill switch. If Core 0 sets this (usually because the SD Card was physically removed or a catastrophic crash occurred), Core 1 instantly zeroes every single volume, flushes all 16 slot pointers, and halts all decoding until the bit is cleared.
- **GCMD_GLOBAL_MUTE (Bit 1):** Instructs Core 1 to apply a target volume of 0 to all Matrix Mixer outputs, naturally fading (5ms anti-pop slew) out all audio without stopping the actual background playback of the files.

This lock-free design allows Core 0 to dictate the intent (Command) while Core 1 independently reports reality (Status), ensuring that the two cores never get deadlocked waiting for each other

2.6 Core 0: System Logistics & Supervision

Core 0 acts as the "Manager," "Bouncer," and "Producer.", it has the following tasks:

- **Filesystem:** Manages the FatFs driver and asynchronous block reads from the MicroSD card.
- **Memory Management:** Acts as the sole allocator for the 8MB external PSRAM "Holding Tank," mapping files into continuous ring buffers.
- **DMA Pumping:** Uses a dedicated high-speed, un-paced Memory-to-Memory DMA channel to blast audio data from PSRAM into Core 1's Tier 4 exchange buffers in microseconds.
- **I^2C Target:** Implements the lock-free command interface for the DCC-EX Command Station via a PIO-based bidirectional I^2C-to-RS422 bridge.
- **Scripting Engine:** Runs the multi-threaded sequence scripting engine, parsing and executing layout automation files (like startup.txt) in the background.
- **Fader Engine:** Calculates and applies background mathematical volume curves (Linear, Logarithmic, S-Curve, Equal Power) for smooth audio panning and fading.
- **CLI & Diagnostics:** Provides a USB-based Command Line Interface for deep debugging, manual control, and an I^2C Wire-Tap analyser that outputs all Command Station traffic in real-time.
- **OLED display:** Drives an optional I^2C OLED display via DMA to provide local, real-time system status, I^2C address, loaded files, and scripting engine status.
- **System Supervisor:** Continuously monitors Core 1's heartbeat and the physical SD Card presence. If the SD Card is removed, it executes a graceful Emergency Stop to safely halt audio without crashing. If a true hardware fault or software panic occurs, it intercepts the crash, logs the ARM register data to the Flash EEPROM, triggers a hardware reboot, and manages the "Three Strikes" Safe Mode recovery.

2.7 Core 1: Deterministic DSP Engine

Core 1 Acts as the "Worker" and "Consumer.", it has the following tasks:

- **Decoding:** Runs 16 instances of the minimp3 decoder.
- **Matrix Mixing:** Performs a 16x8 matrix mix with 32-bit accumulators.
- **Up-sampling:** Executes 2x Zero-Order Hold up-sampling (from 22,050Hz to 44,100Hz).
- **Saturation:** Uses ARMv8-M DSP intrinsics (__ssat) for hardware-level clipping.
- **I^2S Output:** Feeds PIO state machines FIFOs via DMA.

3 Memory Architecture

The design utilizes the RP2350's 10 SRAM banks to ensure Core 1 and Core 0 never compete for the same memory bus.

Region	SRAM	Banks	Role
Logistics RAM	Internal SRAM	0, 1	Core 0 Stack, BSS, Data, and System Dashboard.
PCM Output	Internal SRAM	2	Aligned 8KB Ring Buffers for I2S DMA.
Tier 4 Exchange	Internal SRAM	3, 4	Inter-core data transfer. Core 0 writes, Core 1 reads.
DSP Workspace	Internal SRAM	5, 6, 7	Core 1 Decoder states and scratchpads.
Mass Storage	External PSRAM	N/A	8MB for Sector Maps, Header Caches, and Ring Buffers.
EEPROM	Internal Flash	Final 128KB	Persistently stores I^2C addresses and system config.

Table 2 Memory Architecture

3.1 SRAM Memory Mapping & Bus Isolation

The LSS Engine uses a custom linker script to prevent bus contention by isolating Core 0 and Core 1 into different physical SRAM banks.

Region	Banks	Address Range	Usage
RAM	0-1	0x20000000 - 0x2001FFFF	Core 0 OS, BSS, Active Dashboard
RAM_PCM	2	0x20020000 - 0x2002FFFF	Core 1 -> I^2S Output Ring Buffers
RAM_T4	3-4	0x20030000 - 0x2004FFFF	Inter-Core Tier 4 Exchange
RAM_C1	5-7	0x20050000 - 0x2007FFFF	Core 1 MP3 States & Scratchpads
SCRATCH_X/Y	8-9	0x20080000 - 0x20081FFF	Core Stacks

Table 3 Custom Linker Script SRAM Allocation

4 Control & Communication

4.1 PIO-Based I^2C Bridge

The LSS uses PIO state machines to bridge I^2C over RS422. This provides high noise immunity. It includes a "Wire-Tap" debug queue in PSRAM that records every I^2C transaction for real-time USB monitoring.

4.2 Multi-Byte Writes (File IDs)

When writing the 16-bit File ID to Register 0x20 & 0x21 (or any of the other multi-byte writes, such as the fader registers), the system supports sequential writing. If the Controller sends two bytes in one transaction starting at 0x20, the first byte is stored as the LSB and the second as the MSB. Register 0x21 FILE_ID_LSB and 0x21 FILE_ID_MSB must be written in a single transaction to ensure the RP2350 treats the ID as an atomic (complete) update.
Example:

```
uint16_t file_id = (i2c_reg[0x21] << 8) | i2c_reg[0x20];
```

5 Target I²C Registers

The I²C register map is split into two categories: Global Registers (which apply to the whole system) and Indexed Registers (which apply only to the specific audio slot currently selected by the Command Station).

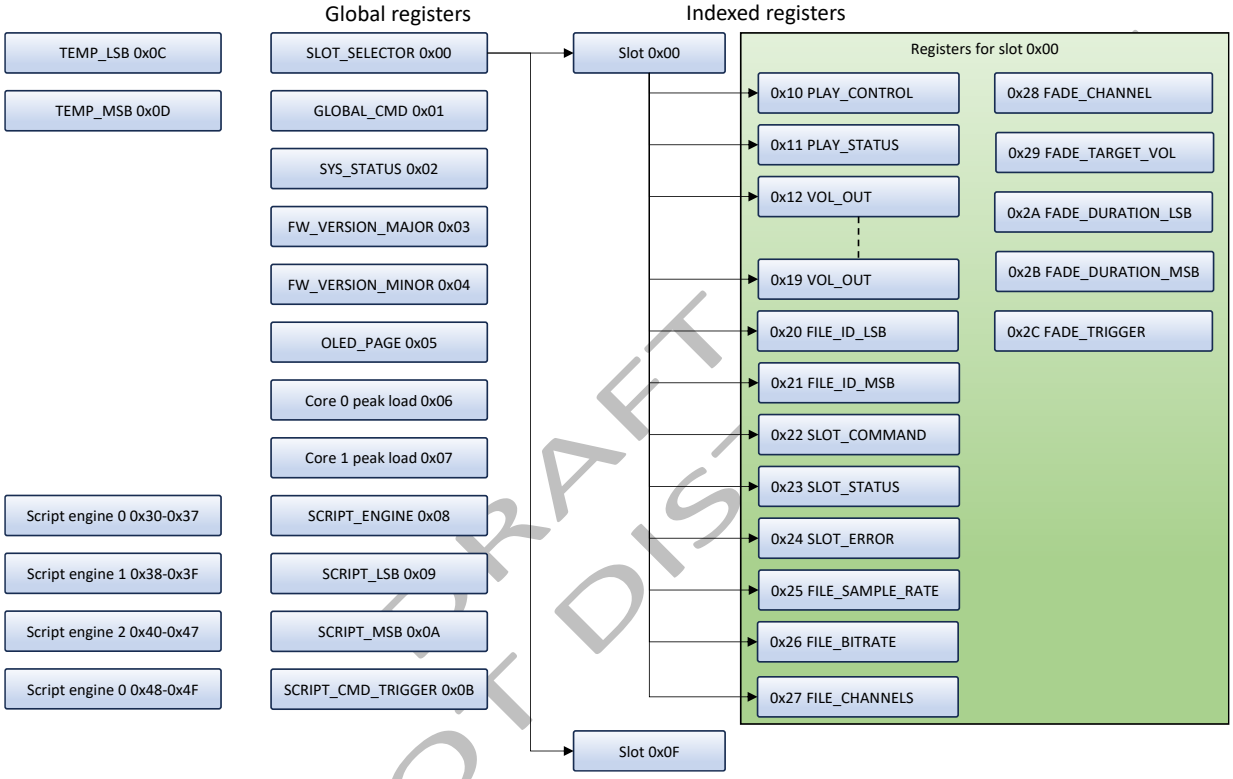

Figure 3 I²C Registers

5.1 Global I²C Registers

These registers control or read system-wide statuses and are not tied to any specific sound file.

Register	Name	Type	Description
0x00	SLOT_SELECTOR	W	Selects active Slot (0–15) for subsequent operations.
0x01	GLOBAL_CMD	W	System-wide commands (Reset, Mute All).
0x02	SYS_STATUS	R	Global hardware status (SD Card, PSRAM, Cores).
0x03	FW_VERSION_MAJOR	R	Firmware version MSB
0x04	FW_VERSION_MINOR	R	Firmware version LSB
0x05	OLED_PAGE	R/W	OLED page display and mode
0x06	CORE0_PEAK_LOAD	R	Core 0 peak load
0x07	CORE1_PEAK_LOAD	R	Core 1 peak load
0x08	SCRIPT_ENGINE	R/W	Script engine selection
0x09	SCRIPT_LSB	R/W	Script ID LSB
0x0A	SCRIPT_MSB	R/W	Script ID MSB
0x0B	SCRIPT_CMD_TRIGGER	R/W	Trigger the script engine to run the script ID
0x0C	TEMP_LSB	R	Internal temperature sensor LSB

0x0D	TEMP_MSB	R	Internal temperature sensor MSB
0x0E-0x0F	RESERVED		Future expansion
0x10	PLAY_CONTROL	R/W	[Indexed] Play, Stop, and Looping triggers.
0x11	PLAY_STATUS	R	[Indexed] Current playback state of the slot.
0x12–0x19	VOL_OUT	R/W	[Indexed] Volume (0–255) for each physical I²S output.
0x20	FILE_ID__LSB	R/W	[Indexed] File Number (0–999) - LSB
0x21	FILE_ID__MSB	R/W	[Indexed] File Number (0–999) - MSB
0x22	SLOT_COMMAND	W	[Indexed] Load/Map or Flush commands.
0x23	SLOT_STATUS	R	[Indexed] Ready/Busy/Error state of the slot.
0x24	SLOT_ERROR	R	[Indexed] Detailed error bitmask (Validation/Decoding).
0x25	FILE_SAMPLE_RATE	R	[Indexed] Detected Sample Rate (bit value).
0x26	FILE_BITRATE	R	[Indexed] Detected Bitrate (kbps).
0x27	FILE_CHANNELS	R	[Indexed] Should always be 1 (Mono)
0x28	FADE_CHANNEL	R/W	[Indexed] Select fading output channel (0-7)
0x29	FADE_TARGET_VOL	R/W	[Indexed] Set volume target (0-255)
0x2A	FADE_ DURATION_LSB	R/W	[Indexed] Fading duration in 10mS increments LSB
0x2B	FADE_ DURATION_MSB	R/W	[Indexed] Fading duration in 10mS increments MSB
0x2C	FADE_TRIGGER	W	[Indexed] trigger the fading, value = fading curve (0-3)
0x2D	RESERVED		Future expansion
0x2E	RESERVED		Future expansion
0x2F	RESERVED		Future expansion
0x30-0x39	SCRIPT_ENGINE 0	R	Scripting engine 0 registers
0x30-0x39	SCRIPT_ENGINE 1	R	Scripting engine 1 registers
0x30-0x39	SCRIPT_ENGINE 2	R	Scripting engine 2 registers
0x30-0x39	SCRIPT_ENGINE 3	R	Scripting engine 3 registers

Table 4 I²C register overview

5.1.1 Register 0x00 Write only SLOT_SELECTOR

Selects the active audio slot (0 to 15) for all subsequent reads and writes to the "Indexed" registers (0x10 through 0x27).

Bit	Name	TYPE	Values
0-7	Output slot	W	0x00 – 0x0F Select the slot. Values grater that 0x0F are ignored

Table 5 Register 0x00 SLOT_SELECTOR

Slot 4-7 are effective if the output expansion board is not installed.
Note: Writing a value greater than 15 (0x0F) is ignored by the implementation.

5.1.2 Register 0x01 Write only GLOBAL_CMD bitmask

Triggers system-wide commands. These act as triggers and are automatically cleared by Core 0 after execution.

Bit	Name	TYPE	Values
0	RESET	W	1 = Sound system reset, flush all cache, ring buffers, I²C command registers, remount microSD card, all default values restored Does not reset I²C to Command Station, this bit is set to 0 after reset or powerup. 0 = Do nothing
1	MUTE_ALL	W	1 = Mute all sound streams, does not affect PLAY_CONTROL or VOL_CH0_CH7 registers, streams output to I²S amplifiers are

			Muted in the matrix mixer. Setting this to 1 instantly silences all output via the matrix mixer's slew-rate limiter without stopping actual playback. 0 = All sound streams are enabled, any VOL_CH0_CH7 values are restored in the matrix mixer
2	OLED_TEST	W	1 = Sent OLED test message to optional OLED display and to USB serial. This bit is set to 0 after the message has been sent 0 = Idle state
3-7	RESERVED		

Table 6 Register 0x01 GLOBAL_CMD

5.1.3 Register 0x02 Read only SYS_STATUS bitmask

Provides a real-time health dashboard of the underlying hardware.

Bit	Name	Values
0	SD_CARD_READY	1 = SD Card is inserted and FatFS is successfully mounted. 0 = SD Card missing or filesystem error.
1	PSRAM_READY	1 = 8MB PSRAM detected and initialized via QMI. 0 = PSRAM failure (system cannot buffer audio)
2	CORE_1_ALIVE	1 = Core 1 (Mixer/Decoder) is running its heartbeat. 0 = Core 1 has crashed or is in reset.
3	CORE_0_ALIVE	1 = Core 0 (Manager/I²C/SD) is running its heartbeat. 0 = Core 0 has crashed or is in reset
4	AUDIO_ENGINE_READY	1 = PIO I²S State Machines & DMA are initialized and running. 0 = Initialization failed (PIO and or DMA resources unavailable)
5	AMP_DETECT	0 = 4 Amps (2x I²S interface) 1 = 8 Amps (4x I²S interface) expansion board installed
6	OLED_DETECTED	1 = OLED display detected 0 = OLED not detected or configured
7	RESERVED	

Table 7 Register 0x02 SYS_STATUS

5.1.4 Register 0x03 Read only FW_VERSION_MAJOR

Byte	Name	Values
0x00	MAJOR	0x00 – 0xFF Major version

Table 8 Register 0x03 FW_VERSION_MAJOR

5.1.5 Register 0x04 Read only FW_VERSION_MINOR

Byte	Name	Values
0x01	MINOR	0x00 – 0xFF Minor version

Table 9 Register 0x04 FW_VERSION_MINOR

5.1.6 Register 0x05 R/W OLED_PAGE

Value	Name	Meaning
0x00	Startup banner	Default
0x01	Auto cycle mode	The screen autonomously pages through the data every 5 seconds
0x02	SYS_STATUS dashboard	System status dash board
0x03	Slots 0-7 files	Files loaded for slot 0-7
0x04	Slots 8-15 files	Files loaded for slot 8-15
0x05	Script Engines	Script engine status, loaded script and last executed script line
0x06		Future expansion
0x07		Future expansion

0x08		Future expansion
0x09		Future expansion
0x0A		Future expansion
0x0B		Future expansion
0x0C		Future expansion
0x0D		Future expansion
0x0E		Future expansion
0x0F		Future expansion

Table 10 Register 0x5 OLED_PAGE

If the OLED display exist, then EX Rail script can set what pre-defined page is displayed, either via a script command, auto cycle mode, or via a push button that trigger the page to be displayed, the default is the startup banned.

5.1.7 Register 0x06 Read only Core 0 peak load

Byte	Name	Values
0x00	Core 0 peak load	0-100

Table 11 Register 0x06 Core 0 peak load in %

Records the peak load of core 0, when read the register will be cleared and populated with the next peak load.

5.1.8 Register 0x07 Read only Core 1 peak load

Byte	Name	Values
0x00	Core 1 peak load	0-100

Table 12 Register 0x07 Core 1 peak load in %

Records the peak load of core 1, when read the register will be cleared and populated with the next peak load.

5.1.9 Register 0x08 R/W SCRIPT_ENGINE

Byte	Name	Values
0x00	SCRIPT_ENGINE	0 = Engine 0 1 = Engine 1 2 = Engine 2 3 = Engine 3

Table 13 Register 0x08 Auto scripting engine selection

5.1.10 Register 0x09 R/W SCRIPT_LSB

Byte	Name	Values
0x00	SCRIPT_LSB	0-999 LSB, 0xFFFF together with MSB 0xFFFF to indicate startup.txt

Table 14 Register 0x09 Automation ScriptID LSB

Register 0x09 (LSB) and 0x0A (MSB) are combined to indicate the scriptID

5.1.11 Register 0x0A R/W SCRIPT_MSB

Byte	Name	Values
0x00	SCRIPT_MSB	0-999 MSB, 0xFFFF together with LSB 0xFFFF to indicate startup.txt

Table 15 Register 0x0A Automation ScriptID MSB

Register 0x09 (LSB) and 0x0A (MSB) are combined to indicate the scriptID

5.1.12 Register 0x0B R/W SCRIPT_CMD_TRIGGER

Byte	Name	Values
0x00	SCRIPT_CMD_TRIGGER	0x01 = Start script ID loaded in registers 0x09 and 0x0A 0x02 = Stop the script it is running

Table 16 Register 0x0B Command trigger to Run or Stop script

5.1.13 Register 0x0C R/O TEMP_LSB

Byte	Name	Values
0x00	TEMP_LSB	

Table 17 Register 0x0C TEMP_LSB

Holds the Least Significant Byte (bits 0–7) of the 16-bit temperature integer. Together with the TEMP_MSB, these two registers form a signed 16-bit two's complement integer representing the temperature in Celsius, scaled by 10.

5.1.14 Register 0x0D R/O TEMP_MSB

Byte	Name	Values
0x00	TEMP_MSB	

Table 18 Register 0x0C TEMP_LSB

Holds the Most Significant Byte (bits 8–15) of the 16-bit temperature integer. Together with the TEMP_LSB, these two registers form a signed 16-bit two's complement integer representing the temperature in Celsius, scaled by 10.

To read and display the correct temperature, the Command Station must execute the following four steps:

1. Read Registers: Read register 0x0C first, then register 0x0D (either sequentially in a single 2-byte transaction, or via separate read transactions).
2. Let the returned values be LSB and MSB.
3. Combine into a Signed 16-bit Integer, shift MSB left by 8 bits and bitwise-OR it with LSB. The result must be stored in a signed 16-bit integer variable (e.g. int16_t in Arduino/C++).
4. Calculate Celsius, divide the combined raw integer by 10.0 to restore the decimal precision.

Formula: raw_value = (MSB << 8) | LSB
Note: Using a signed 16-bit integer type is critical. If the board is running in a cold layout room where the temperature drops below 0 °C, the compiler will automatically handle the negative sign correctly via two's complement conversion.

Example: If the raw integer is 384, the temperature is 38.4 °C. If the raw integer is -15, the temperature is -1.5 °C.

Convert to Fahrenheit (Optional).
If the Command Station is configured for Fahrenheit display, perform the standard conversion on the resulting Celsius value: Formula: Fahrenheit = (Celsius * 1.8) + 32.0

5.2 Indexed I²C Registers (Slot-Specific)

To interact with these registers, the I²C controller (DCC-EX Command Station) must first write the desired slot number (0x00-0x0F) into Register 0x00 (SLOT_SELECTOR).

5.2.1 I²C Playback & Volume Control registers

This section describes the I²C registers handling the play and volume settings for each stream.

5.2.2 Register 0x10 [indexed] R/W PLAY_CONTROL

Controls the playback intent for the selected slot.

Bit	Name	Values
0	STOP/START	1 = Start playing (applies a 5ms anti-pop slew to protect speakers) 0 = Stop playing immediately (applies a 5ms anti-pop slew). (Default)
1	LOOP	1 = Loop the sample indefinitely without gaps 0 = Play sample once and naturally finishes at the EOF (default)
2	PAUSE	1 = Pause playback. Freezes the decoder perfectly in place. (Note: This bit overrides Bit 0 if both are sent) 0 = Pause off
3-7	RESERVED	

Table 19 Register 0x10 [indexed] PLAY_CONTROL

5.2.3 Register 0x11 [indexed] Read only PLAY_STATUS

Reports what the DSP engine is physically doing right now for the selected slot.

Bit	Name	Values
0	IS_PLAYING	1 = Audio is currently streaming to mixer 0 = Idle
1	IS_LOADING	1 = Core 0 is currently reading SD data into 64Kb ring buffer in PSRAM 0 = Finished loading
2	LOOP_ACTIVE	1 = Slot is configured to repeat 0 = Play once
3	SLOT_FAULT	1 = A runtime error occurred (check register 0x24 SLOT_ERROR) 0 = OK
4	IS_PAUSED	1 = Paused 0 = Not paused
5-7	RESERVED	

Table 20 Register 0x11 [indexed] PLAY_STATUS

5.2.4 Register 0x12 through 0x19 VOL_OUT (Read/Write)

Individual output volumes (0-255) routing this slot to the 8 physical hardware speaker channels (0x12 = Channel 0 ... 0x19 = Channel 7).

Register 0x12 [indexed] R/W VOL_OUT Volume (0–255) for physical I²S output

Byte	Name	Values
0x00	VOLUME	0x00-0xFF volume for I²S amplifier output 0 (default 0x00)

Table 21 Register 0x12 [indexed] VOL_OUT channel 0

Register 0x13 [indexed] R/W VOL_OUT Volume (0–255) for physical I²S output

Byte	Name	Values
0x00	VOLUME	0x00-0xFF volume for I²S amplifier output 1 (default 0x00)

Table 22 Register 0x13 [indexed] VOL_OUT channel 1

Register 0x14 [indexed] R/W VOL_OUT Volume (0–255) for physical I²S output

Byte	Name	Values
0x00	VOLUME	0x00-0xFF volume for I²S amplifier output 2 (default 0x00)

Table 23 Register 0x14 [indexed] VOL_OUT channel 2

Register 0x15 [indexed] R/W VOL_OUT Volume (0–255) for physical I²S output

Byte	Name	Values
0x00	VOLUME	0x00-0xFF volume for I²S amplifier output 3 (default 0x00)

Table 24 Register 0x15 [indexed] VOL_OUT channel 3

Register 0x16 [indexed] R/W VOL_OUT Volume (0–255) for physical I²S output

Byte	Name	Values
0x00	VOLUME	0x00-0xFF volume for I²S amplifier output 4 (default 0x00)

Table 25 Register 0x16 [indexed] VOL_OUT channel 4

Register 0x17 [indexed] R/W VOL_OUT Volume (0–255) for physical I²S output

Byte	Name	Values
0x00	VOLUME	0x00-0xFF volume for I²S amplifier output 5 (default 0x00)

Table 26 Register 0x17 [indexed] VOL_OUT channel 5

Register 0x18 [indexed] R/W VOL_OUT Volume (0–255) for physical I²S output

Byte	Name	Values
0x00	VOLUME	0x00-0xFF volume for I²S amplifier output 6 (default 0x00)

Table 27 Register 0x18 [indexed] VOL_OUT channel 6

Register 0x19 [indexed] R/W VOL_OUT Volume (0–255) for physical I²S output

Byte	Name	Values
0x00	VOLUME	0x00-0xFF volume for I²S amplifier output 7 (default 0x00)

Table 28 Register 0x19 [indexed] VOL_OUT channel 7

5.3 I²C Sound File Management & Status registers

5.3.1 Register 0x20 [indexed] R/W FILE_ID_LSB

The Least Significant Byte of the 3-digit file number (0-999) you want to load. Writing here buffers the byte but does not apply it yet.

Byte	Name	Values
0x00	FILE_ID_LSB	0x00-0xFF

Table 29 Register 0x20 [indexed] FILE_ID_LSB

5.3.2 Register 0x21 [indexed] R/W FILE_ID_MSB

The Most Significant Byte of the file number. Writing this register atomically combines it with the buffered LSB and sets the slot's target File ID.

Byte	Name	Values
0x00	FILE_ID_MSB	0x00-0xFF

Table 30 Register 0x21 [indexed] FILE_ID_MSB

5.3.3 Register 0x22 [indexed] Write Only SLOT_COMMAND

Instructs Core 0 to perform logistics actions on the slot.

Bit	Name	Values
0	LOAD	1 = Load sound sample file, build PSRAM sector map, validate the MP3 headers, fill header cache and ring buffer, file name assigned in registers 0x20/0x21 FILE_ID 0 = Idle, bit set to 0 after Load execution (default)
1	FLUSH	1 = Instantly kills playback, flush the header cache and ring buffers, unassigns the file, and register 0x11 is cleared. 0 = Idle, bit set to 0 after Flush execution
2-7	RESERVED	

Table 31 Register 0x22 [indexed] SLOT_COMMAND

5.3.4 Register 0x23 [indexed] Read only SLOT_STATUS

Reports the logistics/loading state of the slot.

Bit	Name	Values
0	EMPTY	1 = Sample assigned 0 = No sample assigned (default)
1	BUSY	1 = Currently fetching next data from microSD 0 = Idle (default)
2	READY	1 = Mapped, validated, and primed in PSRAM, ready to play 0 = Not ready to play (default)
3	ERROR	1 = Loading or validation failed. Check Register 0x24 SLOT_ERROR 0 = No error (default)
4-7	RESERVED	

Table 32 Register 0x23 [indexed] SLOT_STATUS

5.3.5 Register 0x24 [indexed] Read only SLOT_ERROR bitmask slot error

A detailed bitmask explaining exactly why a slot faulted or failed to load

Bit	Name	Description
0	FILE_NOT_FOUND	No file matching the 3-digit ID exists on the SD card
1	RATE_MISMATCH	File is not 22050Hz (detected during pre-load)
2	BITRATE_MISMATCH	File is not 128kbps (detected during pre-load)
3	VBR_DETECTED	Strict Mode: Bitrate changed mid-stream (Variable Bit Rate)
4	FORMAT_CHANGE	Sample rate changed mid-stream
5	STEREO_ERROR	File is Stereo. System requires Mono
6	SYNC_ERROR	Malformed MP3 data or corrupted file, or buried in excessive ID3 tags
7	SD_ERROR	SD Card read timeout or hardware failure, this is a catastrophic failure

Table 33 Register 0x24 [indexed] SLOT_ERROR

If there is a low level SD_ERROR, the recommended action is to replace the microSD card.

5.3.6 Register 0x25 [indexed] Read only FILE_SAMPLE_RATE

Returns the detected sample rate enum (1 = 22050 Hz, which is the only supported value right now)

Byte	Name	Values
0x00	FILE_SAMPLE_RATE	0 = Unknow, default / no file loaded 1 = 22,050 Hz, system standard 2 = 44,100 Hz, not supported 3 = 48,000 Hz, not supported 4 = 32,000 Hz, not supported 5 = 11,025 Hz, not supported

Table 34 Register 0x25 [indexed] FILE_SAMPLE_RATE

5.3.7 Register 0x26 [indexed] Read only FILE_BITRATE

Returns the actual detected bit rate in kbps (e.g., 128).

Byte	Name	Values
0x00	FILE_BITRATE	Actual value of the bit rate in binary, should read 128, set to 1 if VBR is detected (default is 0 no file loaded)

Table 35 Register 0x26 [indexed] FILE_BITRATE

5.3.8 Register 0x27 [indexed] Read only FILE_CHANNELS

Returns the number of channels detected (1 = Mono).

Byte	Name	Values
0x00	FILE_CHANNELS	0x00 = no channels detected (default when no file loaded) 0x01 = 1 Channel detected (Mono) 0x02 = 2 Channels detected (Stereo)

Table 36 Register 0x27 [indexed] FILE_CHANNELS

5.4 I²C Fading registers

The Fader Engine is a dedicated background process running on Core 0 that handles smooth, mathematically precise volume transitions without blocking the main audio pipeline or the Command Station.
Rather than forcing the Command Station to rapidly spam the I²C bus with hundreds of individual volume updates to create a fade, the LSS Engine handles the heavy lifting internally.

To initiate a fade, the Command Station (EX Rail script) must write to all five fading registers (0x28 through 0x2C). Registers 0x28 through 0x2B act as a setup cache to define the target physical output channel, target volume, and fade duration. Finally, writing to Register 0x2C selects the desired mathematical curve (Linear, Logarithmic, S-Curve, or Equal Power) and acts as the "atomic trigger" that actually starts the transition.

This ‘cache-then-trigger’ design guarantees that all parameters are safely locked in before the fade begins, enabling effortless "fire-and-forget" audio illusions, for example like smooth ambient cross-fades or high-speed Doppler train flybys.

5.4.1 Register 0x28 [indexed] RW FADE_CHANNEL

The physical output speaker (0-7) you want to fade

Byte	Name	Values
0x00	FADE_CHANNEL	0-7

Table 37 Register 0x28 [indexed] FADE_CHANNEL

5.4.2 Register 0x29 [indexed] RW FADE_TARGET_VOL

The final volume to reach (0-255)

Byte	Name	Values
0x00	FADE_TARGET_VOL	0-255

Table 38 Register 0x29 [indexed] FADE_TARGET_VOL

5.4.3 Register 0x2A [indexed] RW FADE_ DURATION_LSB

Lower 8 bits of the fade time.

Byte	Name	Values
0x00	FADE_ DURATION_LSB	0x00-0xFF (0-255)

Table 39 Register 0x2A [indexed] FADE_DURATION_LSB

5.4.4 Register 0x2B [indexed] RW FADE_ DURATION_MSB

Upper 8 bits of the fade time

Byte	Name	Values
0x00	FADE_ DURATION_MSB	0x00-75 (0-117)

Table 40 Register 0x2B [indexed] FADE_DURATION_MSB

Register 0x2A and 0x2B are combined into a 16 bit unsigned integer, the combined value is from 1-30,000 and represent the fading time in 10 milli-second intervals, e.g., 400 = 4 seconds.

5.4.5 Register 0x2C [indexed] Write only FADE_TRIGGER

Writing to this register instantly executes the fade using the cached parameters.

Byte	Name	Values
0x00	FADE_TRIGGER	0x00 = Linear 0x01 = Logarithmic 0x02 = S-Curve 0x03 = Equal power

Table 41 Register 0x2C [indexed] FADE_TRIGGER

5.5 I²C Scripting engine telemetry registers

The I²C Scripting Engine Telemetry registers (0x30 through 0x4F) provide the Command Station with a continuous, real-time status window into the EX_Iss Engine's background automation system.

Because the four multi-threaded sequence engines run independently on Core 0, the Command Station needs a way to monitor their progress without stalling its own processor or interrupting the audio pipeline.

These strictly read-only registers are divided into four contiguous 8-byte blocks, one for each script engine. By polling these blocks, the Command Station can instantly determine an engine's current execution state (such as IDLE, EXEC, or WAIT_TIMER), verify exactly which script ID is loaded, and track the specific line number currently being evaluated. This allows layout builders to effortlessly synchronize external DCC events with a script's internal sequences.

For example, a user could have a background audio script that plays a localized thunderclap, and the exact millisecond the script Engine evaluates line 45 of the script(the thunder sound), the EX-Rail script sees the CURRENT_LINE register hit 45 and instantly flashes the layout's LED room lights! The Command Station doesn't have to keep track of audio durations; it just polls the "audio sensor" and reacts.

5.5.1 Registers 0x30 – 0x4F Read only SCRIPT_ENGINES Telemetry

Script engines run scripts, we have 4 script engines to run 4 scripts concurrently. Each script engine has 8 Bytes.

Offset	Name	Values
0x00	STATE	0x00 = IDLE, not running a script 0x01 = EXEC, executing a script 0x02 = WAIT_TIMER, waiting for a timer to expire 0x03 = WAIT_SLOT, waiting for an audio slot to finish playing 0x04 = WAIT_LOAD, waiting for the SD card to load a file 0x05-0xFF Reserved for future
0x01	SCRIPT_ID_LSB	The lower 8 bits of the currently loaded 3-digit script ID (0-999)
0x02	SCRIPT_ID_MSB	The upper 8 bits of the currently loaded 3-digit script ID
0x03	CURRENT_LINE_LSB	The lower 8 bits of the line number the script is currently evaluating or waiting on
0x04	CURRENT_LINE_MSB	The upper 8 bits of the current line number
0x05	LAST_FINISHED_LINE_LSB	The lower 8 bits of the most recently completed line number
0x06	LAST_FINISHED_LINE_MSB	The upper 8 bits of the most recently completed line number
0x07	RESERVED	Always return 0x00

Table 42 Register 0x30-0x4F Scripting Engines telemetry

Scripting engine telemetry I²C address ranges:
Engine 0 register 0x30-0x37
Engine 1 register 0x38-0x3F
Engine 2 register 0x40-0x47
Engine 3 register 0x48-0x4F

6 User interface CLI commands

6.1 Introduction

The USB Serial command interpreter uses a framed command format similar to the DCC-EX Command Station. All commands must be surrounded by `<` and `>` to indicate the beginning and end of a command.

Command syntax:

1. Op-codes (the first letter, e.g., `D` for Data/Write, `R` for Read) are case sensitive.
2. Keywords (e.g., ` SYS `, `SLOT`) are case sensitive and must be specified exactly.
3. Parameters are separated by a single space ` `.
4. Square brackets `[...]` denote variables you must provide (do not type the brackets).
5. The pipe character `|` denotes "OR" (pick one of the options).

Example:
<D SLOT 0 LOAD 12> -> Loads File ID 12 into Slot 0.

One of the main advantages of having a command interpreter is that it enable testing of sound samples, testing scripts and diagnostics of the audio engine before committing a DCC EX script.

6.2 Global Write Commands (<D SYS ...>)

These commands affect the entire LSS Engine system.

<D help>
Prints a quick-reference help menu to the USB serial terminal.

<D SYS reset 1>
Triggers a soft system reset. Flushes all caches, destroys ring buffers, clears I2C command registers, unloads all slots, and remounts the SD card.
Does not reset the physical I2C connection to the Command Station.

<D SYS mute 1|0>
1 = Global Mute ON. Instantly fades out all audio output via the matrix mixer.
Does not stop active playback (files keep running silently in the background).
0 = Global Mute OFF. Naturally fades audio back to their configured volume levels.

<D SYS oled test 1>
Queues an OLED test message. (Currently logs to USB Serial, reserved for future display driver).

<D SYS i2c debug 1|0>
1 = Enables the I^2C Debug Wire-Tap. Allocates a 64KB PSRAM circular buffer to capture and print every single I^2C Read/Write transaction between the LSS and the Command Station.
0 = Disables the I^2C Debug Wire-Tap.

Note: enabling the I^2C Debugging can create a large amount of data. As a result of a possible overrun the newest messages are dropped when the USB output queue is full.

<D SYS i2c address [0x08 - 0x77]>
Saves a new base I²C address to the RP2350's internal Flash EEPROM and instantly reboots the system to apply it. Accepts standard integers or hex (e.g., 0x40).

<D SYS oled page 0-5>
Change OLED Kiosk Page (0=Auto, 1=Startup, 2=Sys, 3=Slots 0-7, 4=Slots 8-15, 5=Script Engines.

<D SYS factory reset>
Wipes the internal Flash EEPROM back to factory defaults (restoring I²C base address 0x40) and instantly reboots the system.

<D SYS clear crash>
Clears any saved post-mortem crash logs from the Flash EEPROM. This preserves your configured I²C address and OLED settings.

<D config oled [width] [height]>
Configures the OLED display and instantly reboots to apply the settings. This will be saved to internal Flash EEPROM.
[width] = 0 (Disabled), 128 (SSD1306), 132 (SH1106)
[height] = 32 or 64

<D RUN SCRIPT [id] [engine_id]>
Executes a sequence script in the specified background scripting engine.
[id] = The 3-digit numeric ID of the script (e.g., 001 for 001_sequence.txt).
If omitted, the engine will run 'startup.txt'.
[engine_id] = Optional. The engine to run the script on (0 to 3). Defaults to 0.

The script file naming is [3 digit id]*.txt, the system expects a 3-digit number (from 000 to 999) at the very beginning of the file name, and it must end with the .txt extension

Note: Be careful when running multiple scripts concurrently on different engines.
If two scripts attempt to control the same Slot ID (e.g., using LOAD or FLUSH) at the same time, they will interrupt and override each other's audio.

<D STOP SCRIPT [engine_id]>
Stops script execution on the specified scripting engine (0 to 3).

<D SCRIPT debug verbose 1|0>
1 = Enables verbose scripting engine logging to the CLI.
0 = Disables verbose logging. Only critical script execution errors will be printed (default).

6.3 Slot Write Commands (<D SLOT [slot] ...>)

These commands control individual audio slots. [slot] must be an integer from 0 to 15.

<D SLOT [slot] LOAD [file_id]>

Instructs Core 0 to locate the 3-digit [file_id] (0-999) on the SD card, validate its MP3 headers, build the PSRAM sector map, and pre-load the header cache and 64KB ring buffer.
Example of valid values are 0, 01 or 001, the search pattern will be “001*”

The file format on the microSD card must start with a number between 000-999. Everything after that is for the user to easier identify a sound sample, examples:
xxx<your filename>, xxx_<your filename>, xxx-<your filename>, xxx <your filename>
The file extension is ignored, the system checks if a file is a genuine MP3 file before loading.

<D SLOT [slot] FLUSH>

Instantly stops playback on the slot, destroys its ring buffer, and unassigns the file.

<D SLOT [slot] OUT [channel] [volume]>

Routes the slot's audio to a specific physical amplifier channel.
[channel] = 0 to 7 (Represents the 8 physical hardware speakers).
[volume] = 0 to 255 (0 is silent, 255 is 100% volume).

<D SLOT [slot] PLAY CONTROL [flags]>

Controls playback state. Flags can be combined.
Flags:
P = Play (Fades audio in and begins decoding).
S = Stop (Fades audio out and halts). 'S' overrides 'P' if both are sent.
U = Pause (Halts playback but retains current position).
L = Loop (Plays the file continuously without gaps).

Examples:
<D SLOT 0 PLAY CONTROL P L> -> Plays Slot 0 in an infinite loop.
<D SLOT 0 PLAY CONTROL S> -> Stops Slot 0.

<D SLOT [slot] FADE [channel] [target_volume] [duration_ticks] [curve]>

Smoothly transitions the volume of a specific output channel in the background.
This command is NON-BLOCKING; the script will instantly proceed to the next line while the fade happens. To wait for the fade to finish, use a <D WAIT> command.

```
[channel]        = The output speaker channel (0 to 7).
[target_volume]  = The final volume to reach (0 to 255).
[duration_ticks] = The length of the fade in 10ms increments (1 to 30000). e.g., 400 = 4
                   seconds.
[curve]          = The fade shape: 'LINEAR' (constant speed), 'LOG' (natural audio taper),
                   'S-CURVE'(flyby/doppler), or 'EQUAL_POWER' (ambient cross-fade).
```

Type of fader curves:

LINEAR - The "Mechanical" Baseline

Best for: Exact mathematical control or abrupt mechanical stops.

Why it fits: Sometimes you don't want audio to sound "natural"; you want it to sound mechanical. A Linear curve is great for things like a sawmill machine powering down, or just as a predictable baseline if the user is trying to stack multiple commands together to build their own custom audio sample.

LOG (Logarithmic / Audio Taper) - The "Distance" Curve

Best for: Fading sounds in or out over a distance, or general ambient fades.
Why it fits: Because human hearing is logarithmic, this curve makes a fade sound natural. If a layout builder wants a train to sound like it is slowly approaching from miles away, or if they want to smoothly fade in the background "city traffic" loop over 10 seconds without jarring the listener, this is the go-to curve.

S-CURVE (Sigmoid / Smooth step) - The "Flyby" Curve

Best for: High-speed panning (moving a train from one speaker to another).
Why it fits: As with the Doppler effect, this curve perfectly mimics the geometry of a moving object passing a stationary observer. If a user pans a locomotive whistle from Speaker 1 to Speaker 2 using an S-Curve, it will linger in Speaker 1, whip violently through the centre, and linger in Speaker 2. It is the absolute best way to simulate a train physically moving past the audience.

EQUAL POWER (Square Root) - The "Ambient Cross-fade" Curve

Best for: Shifting environmental states or centre stationary sounds.
Why it fits: If a user has a "Daytime Birds" loop playing and wants to cross-fade it into a "Nighttime Crickets" loop as the layout room lights dim, using a Linear or Log curve will cause the overall volume of the layout to noticeably dip in the exact middle of the transition. An Equal Power curve pushes the volume up slightly in the middle, ensuring the room's total acoustic energy stays perfectly flat during the transition.

Example Pan (Cross-fade from speaker 0 to speaker 1 over 4 seconds):
<D SLOT 0 FADE 0 0 400 LOG>
<D SLOT 0 FADE 1 255 400 LOG>
<D WAIT 4000>

6.4 Status & Read Commands (<R ...>)

These commands query the system and return formatted diagnostic data to the terminal.

<R SYS STATUS>
Returns a comprehensive global health dashboard.
Output includes:

- Active I^2C Address.
- SD Card hardware status (READY / MISSING / MOUNT FAILED).
- PSRAM memory hardware status.
- Core 0 and Core 1 heartbeat status (including Core 0 and 1 CPU Load %).
- Audio Engine PIO/DMA initialization status.

- Detected hardware amplifiers (4 default, 8 with expansion board).
- OLED Display detection status.
- Global Mute status.
- Script Engines:

```
-------------------------------------------------------
Engine | State    | Script   | Line | Last Finished
-------------------------------------------------------
     0 | IDLE     | startup  |    - |             -
     1 | IDLE     | 001      |    - |             -
     2 | IDLE     | empty    |    - |             -
     3 | IDLE     | empty    |    - |             -
-------------------------------------------------------
```

<R FW>

Returns the firmware version (Major.Minor).

<R SHOW FILES>

Iterates through all 16 slots and prints a list of every valid file currently loaded, including the Slot ID, the parsed 3-digit File ID, and the full original filename.

<R SHOW FILES [slot]>

Returns the File ID and full filename loaded into the specific [slot].

<R SLOT [slot] PLAY CONTROL>

Returns the current human-readable playback state of the slot.

Possible Returns:

- "Slot [x] is empty"
- "Slot [x] file [id] playing"
- "Slot [x] file [id] playing in Loop"
- "Slot [x] file [id] stopped playing"

<R SLOT [slot] STATUS>

Returns a highly detailed diagnostic dump for a specific slot, identical to what the Command Station sees via I2C.

Output includes:

- Raw hex values for SLOT_STATUS (0x23) and SLOT_ERROR (0x24).
- Decoded MP3 Format Information: Sample Rate (Hz), Bitrate (kbps), and Channels.
- Decoded human-readable Error Flags if the slot failed, such as:

1. FILE_NOT_FOUND
2. RATE_MISMATCH (Must be 22050Hz)
3. BITRATE_MISMATCH (Must be 128kbps)
4. VBR_DETECTED (Strict enforcement tripped)
5. FORMAT_CHANGE (Sample rate shifted mid-stream)
6. STEREO_ERROR (Must be Mono)
7. SYNC_ERROR (Corrupt file or buried in metadata)
8. SD_ERROR (Hardware/DMA read timeout)

<D SHOW REGISTERS>

Show all Global I²C registers

```
CLI: Global I²C Registers example output
  [0x01] GLOBAL CMD: 0x00
  [0x02] SYS STATUS: 0xBF
  [0x03] FW VER MAJ: 0x00
  [0x04] FW VER MIN: 0x17
  [0x08-0x0B] AUTO CMD: Engine=0, Script=0x0000, Trigger=0x00
  [0x30-0x4F] SCRIPT ENGINES:
    ------------------------------------------------------
    Engine | State | Script   | Line | Last Finished
    ------------------------------------------------------
         0 |     0 | startup  |      6 |          6
         1 |     0 | 001      |      6 |          6
         2 |     0 | empty    |      0 |          0
         3 |     0 | empty    |      0 |          0
    ------------------------------------------------------
```

<D SHOW REGISTERS [slot]>

Show Indexed I²C registers for a specific slot

```
CLI: Slot 0 Indexed Registers example output
  [0x10] PLAY CONTROL: 0x00
  [0x11] PLAY STATUS : 0x00
  [0x12-0x19] VOL OUT: 32 00 00 00 00 00 00 00
  [0x20-0x21] FILE ID: 002 (0x0002)
  [0x22] SLOT CMD    : 0x00
  [0x23] SLOT STATUS : 0x05
  [0x24] SLOT ERROR  : 0x00
  [0x25] SAMPLE RATE : 0x01
  [0x26] BITRATE     : 0x80
  [0x27] CHANNELS    : 0x01
  [0x28-0x2B] FADER CACHE: Ch=0, TgtVol=0, Dur=0ms
```

<D config oled [width] [height]>

Set OLED Width (0=Disabled, 128=SSD1306, 132=SH1106) and Height (32|64)
Default is Type: 128 and Height: 64

<D SHOW CRASH DATA>

Displays a comprehensive diagnostic report containing the Processor model, unique hardware Board ID, Firmware version, and any post-mortem crash data (e.g., Program Counter, Link Register) recovered from Flash memory after a hard fault or watchdog reset. Extremely useful for field support and warranty verification. After a fatal crash has occurred, the system saves the crash data and reboots in safe mode.

<D SYS clear crash>

Clears any saved post-mortem crash logs from the Flash EEPROM. This preserves the configured I²C address and OLED settings.
This command MUST be entered in the CLI when the system is in ‘safe mode’ to ‘unlock’ the system.

<D SHOW MIXER>

Displays a global overview of the matrix mixer, including active output channels, number of active streams per channel, and peak target volume.

Example output:

```
CLI: Matrix Mixer Global Statistics
Global Mute: OFF
-----------------------------------------------------
Out Ch | Status       | Active Streams | Peak Target Vol
-----------------------------------------------------
         0 | ACTIVE   |              2 | 255 (100%)
         1 | ACTIVE   |              1 | 128 ( 50%)
         2 | IDLE     |              0 |   0 (  0%)
         3 | IDLE     |              0 |   0 (  0%)
         4 | IDLE     |              0 |   0 (  0%)
         5 | IDLE     |              0 |   0 (  0%)
         6 | IDLE     |              0 |   0 (  0%)
         7 | IDLE     |              0 |   0 (  0%)
-----------------------------------------------------
Total active cross-points: 3 / 128
```

Total active cross-points: Indicates the total number of slot-to-output intersections actively mixing audio compared to the maximum possible matrix capacity (16 slots × 8 outputs = 128 possible intersections).

<D SHOW MIXER [0-7]>

Displays detailed mixer statistics for a specific output channel, showing which streams are routed, their target and actual slew volumes, and current status.

Example output:

```
CLI: Matrix Mixer Details for Output Channel 0
State: ACTIVE (2 streams mixing)
-------------------------------------------------------------------
Slot | File ID | Target Vol | Actual Slew | Status
-------------------------------------------------------------------
   0 |     001 |        255 |         255 | PLAYING (Loop)
   1 |     002 |          0 |          48 | FADING OUT (Slewing)
   3 |     015 |        128 |           0 | PAUSED
-------------------------------------------------------------------
```

Output Breakdown:

State: Indicates if the output channel is currently outputting sound (ACTIVE), primed but not playing (IDLE (No active streams, but some are routed)), or has no assignments (IDLE (No streams routed)).
Table Columns:
Slot: The slot ID (0 to 15).
File ID: The 3-digit ID of the file loaded into the slot (or --- if empty/invalid).
Target Vol: The user-defined matrix target volume (0–255) for this slot-channel combination.
Actual Slew: The current, real-time volume level (0–255) as calculated by Core 1's slew-rate limiter.
Status: The current state of the slot:
PLAYING (Loop): Stream is active and loops infinitely.
PLAYING: Stream is active and will play once.
FADING OUT (Slewing): Target volume is 0 (or stream reached EOF), but the volume is ramped down by the slew limiter and has not hit zero yet.
PAUSED: The stream is paused.
MUTED: Global mute is active.
ERROR: The slot has encountered a hardware/format error.
IDLE: Loaded and routed, but not playing.
STOPPED: Not playing and slew rate volume has reached zero.

7 Scripting Commands

The design philosophy is that scripts are entirely isolated to audio manipulation. They can load, play, fade, route, and stop audio within the 16 slots, but they have zero access to system administration, diagnostics, or other scripting engines.

The following commands are exclusively available inside sequence scripts (e.g., startup.txt or 001_sequence.txt). If typed into the USB CLI, they will be rejected to prevent blocking the interactive terminal.

<D WAIT [ms]>

Pauses the sequence script for the specified number of milliseconds.
Range: 0 to 300,000 milliseconds

<D WAIT RANDOM [min_ms] [max_ms]>

Pauses the sequence script for a genuinely unpredictable duration between [min_ms] and [max_ms]. Powered by the RP2350's true hardware random number generator.
Range:
Minimum wait (min_ms): Must be greater than or equal to 0.
Maximum wait (max_ms): Must be strictly greater than the minimum wait (max_ms > min_ms) and less than or equal to 300,000 milliseconds (5 minutes)

<D WAIT SLOT [slot] FINISHED>

Pauses the sequence script until the specified audio slot finishes playing naturally (EOF fading complete) or until the slot throws a hardware error.

<D SLOT [slot] PLAY [count]>

Plays the slot for exactly [count] times before automatically stopping. Example: <D SLOT 0 PLAY 3> plays the file three times in a row.
(This replaces PLAY CONTROL P/L/S inside sequence scripts).
Range:
1: Plays the sound once.
255: Plays the sound 255 times sequentially

7.1 Invalid Script Commands

The following commands are blocked from being used in a script:
Scripts are strictly sandboxed and **cannot** run the following commands:

- **System & EEPROM Commands (<D SYS ...>, <D config ...>)**: Scripts are blocked from resetting the system, changing OLED pages, formatting the crash log, or modifying EEPROM settings (like I^2C addresses). This prevents accidental infinite reboot loops and protects the Flash memory from being worn out by continuous writes.
- **Read & Diagnostic Commands (<R ...>, <D SHOW ...>, <D help>)**: Scripts execute in the background and don't have a user looking at a terminal. Running read commands or dumping registers would just silently spam the system and waste CPU cycles, so they are blocked.
- **Script Control Commands (<D RUN SCRIPT>, <D STOP SCRIPT>)**: Scripts are not allowed to call, spawn, or stop other scripts. This prevents "script inception" (scripts launching scripts

launching scripts) which would quickly cause stack overflows or infinite execution loops across the 4 scripting engines.

7.2 Valid CLI Commands for Scripts

Aside from the script-only commands mentioned (WAIT, WAIT RANDOM, WAIT SLOT FINISHED, and SLOT PLAY [count]), scripts are only allowed to execute Slot Manipulation commands:

- <D SLOT [x] LOAD [y]>
- <D SLOT [x] FLUSH>
- <D SLOT [x] OUT [ch] [vol]>
- <D SLOT [slot] FADE [channel] [target_volume] [duration_ticks] [curve]>
- <D SLOT [x] PLAY CONTROL [flags]>

8 Crash Data Logging and Recovery Architecture

This section details how CPU Hard Faults and Hardware Watchdog resets are detected, recorded, and handled within the LSS Engine firmware.

8.1 Storage Layout in Flash EEPROM

The last sector of Flash (EEPROM_BASE_ADDR) stores the persistent SystemConfig struct. We added diagnostic fields to this sector to record crashes:

- crash_magic : Set to 0xDEADFACE when a crash has occurred.
- crash_count : The consecutive strike counter (increments on back-to-back crashes).
- crash_pc : Program Counter (address of the instruction that caused the crash).
- crash_lr : Link Register (return address / function call register).
- crash_sp : Stack Pointer (stack address at the time of the crash).
- crash_core : The CPU core ID that crashed (0, 1, or 255/0xFF for Watchdog).

8.2 CPU Hard Fault Interception (VTOR Redirection)

By default, the Pico SDK registers exclusive handlers that loop indefinitely on a crash. We bypass this to record post-mortem registers:

A. The RAM Vector Table:

- During boot (before Core 1 is launched), Core 0 copies the vector table from Flash to a 512-byte aligned RAM array called `ram_vtable`.
- It overwrites vector slot 3 (Hard Fault Exception) with `my_hard_fault_handler`.
- Core 0 writes the RAM table address to its private `VTOR` register.
- When Core 1 starts up, it also points its private `VTOR` register to the same global `ram_vtable` array.

B. Exception Capture Flow:

- When a Hard Fault occurs on either Core 0 or Core 1:

1. The CPU jumps to `my_hard_fault_handler` (a naked assembly wrapper).
2. The wrapper determines if the CPU was using the Main Stack (MSP) or Process Stack (PSP), extracts the Stack Pointer (SP), and calls `my_hard_fault_handler_c(uint32_t* sp)`.
3. The C handler halts Core 1 (`multicore_reset_core1()`) to prevent concurrent Flash access.
4. It extracts the PC and LR directly from the ARM exception frame stored on the stack.
5. It reads the current config from Flash, increments the crash strike counter, saves the registers and core ID, and writes them back to the Flash sector.
6. It triggers an immediate watchdog reboot to restart the board.

Output example:

```
<D SHOW CRASH DATA>\r" ----
CLI: Diagnostic Crash Report
=====================================
Firmware Version : 0.24A4
Processor Model  : RP2350 (Silicon Rev 2)
Hardware Board ID: CA501CBD55CC969F
```

```
System Clock     : 150 MHz
--------------------------------------
Crash Log Sector : CRASH DETECTED! (Strike Count: 3)
  - Crashed Core : 1
  - Prog Counter : 0x10003030
  - Link Register: 0x10028264
  - Stack Pointer: 0x2007C248

  To locate the exact line of code, run:
  arm-none-eabi-addr2line -e EX_lss.elf 0x10003030
  on your development machine.
======================================
```

8.3 Hardware Watchdog Resets (0xDEADBEEF)

- If Core 1 hangs in a loop or Core 0 stops feeding the hardware watchdog:
- The hardware watchdog timer (configured for 5 seconds) expires and resets the RP2350 chip.
- During early boot (before Core 1 is launched), Core 0 checks:`watchdog_caused_reboot()`
- If true, Core 0:
  1. Reads the current config from Flash.
  2. Increments the crash strike counter.
  3. Writes `0xDEADBEEF` to the PC, LR, and SP fields in Flash (since there is no stack frame to extract registers from).
  4. Sets the core ID to 255 (0xFF).
  5. Writes the updated config back to the Flash sector.

Output example:

```
<D SHOW CRASH DATA>\r" ----
CLI: Diagnostic Crash Report
======================================
Firmware Version : 0.24A4
Processor Model  : RP2350 (Silicon Rev 2)
Hardware Board ID: CA501CBD55CC969F
System Clock     : 150 MHz
--------------------------------------
Crash Log Sector : CRASH DETECTED! (Strike Count: 2)
  - Crashed Core : 255
  - Prog Counter : 0xDEADBEEF
  - Link Register: 0xDEADBEEF
  - Stack Pointer: 0xDEADBEEF

  To locate the exact line of code, run:
  arm-none-eabi-addr2line -e EX_lss.elf 0xDEADBEEF
  on your development machine.
======================================
```

8.4 The "Three Strikes" Safe Mode Safety Net

To prevent the board from entering an infinite boot loop that locks up the CPU:

- Every time the system boots, it checks the strike counter in Flash.
- If the counter is 3 or more (consecutive crashes/watchdogs), it enters SAFE MODE:
  - Core 1 is NOT launched (remains halted).

  - Audio and mixing are disabled.
  - A fatal warning banner is printed to the CLI console.
  - The CLI stays active so you can inspect the registers or clear the logs.
- Collect the crash data with the **<D SHOW CRASH DATA>** command via the USB CLI.
- Recovery: You must run `**<D SYS clear crash>**` to reset the strike counter and enable the audio engine.

8.5 Automatic Strike Counter Reset

If the board boots successfully and runs stably for 60 seconds without a crash:

- A stability timer resets the strike counter in Flash back to 0.
- This ensures that sporadic crashes separated by long periods of healthy operation do not accumulate and trigger Safe Mode.